COMPUTER ENGINEERING

QUEZON CITY

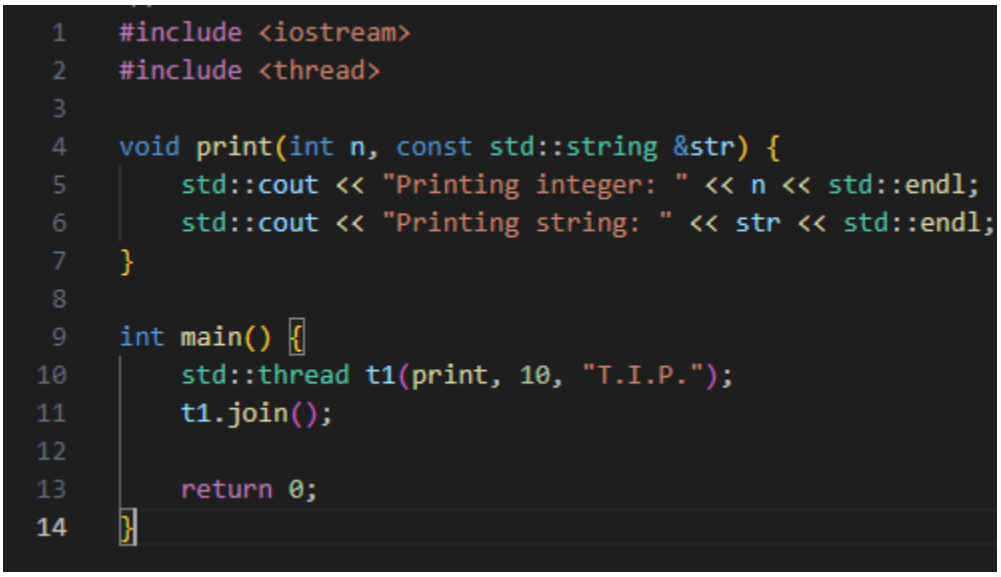
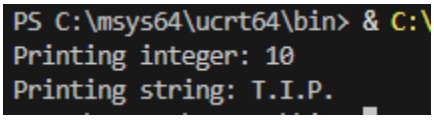
COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10-16-2026	**DATE SUBMITTED:**	10–20-2026
NAME:	Condino, Niel Vincent B. Guarin, Jan Mikaela C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

Part 1: Simple One-Threaded Example

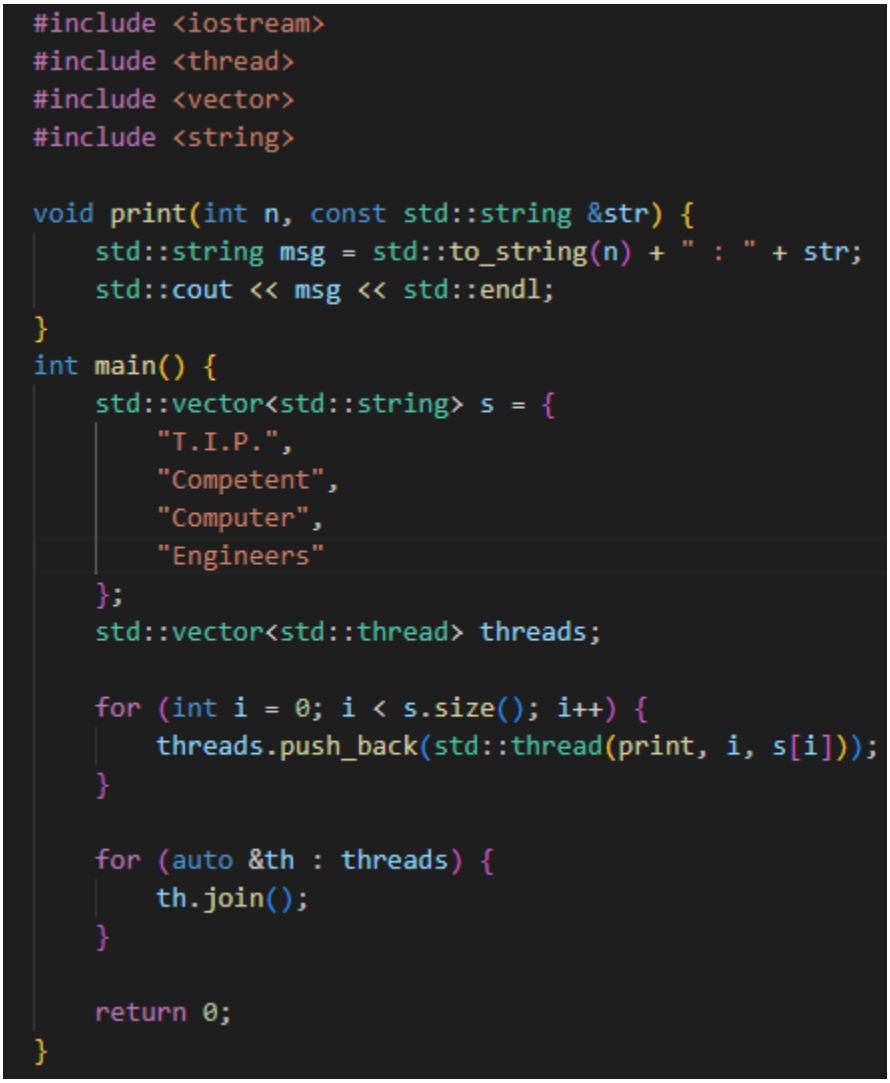
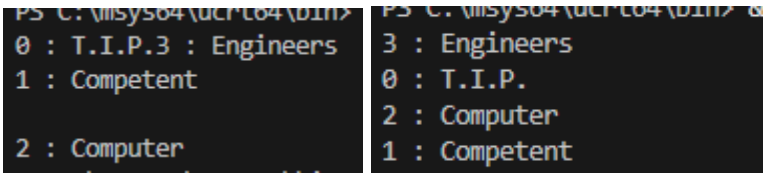
Table 13-1. Simple One-Threaded Example

Code Screenshot	
Output Screenshot	
Analysis	The program creates one thread named **t1** to execute the **print()** function. It passes the integer "10" and the string "T.I.P." as arguments. The **join()** function makes the main program wait until the thread finishes executing to ensure that the output is completed before the program terminates.

```
1   #include <iostream>
2   #include <thread>
3
4   void print(int n, const std::string &str) {
5       std::cout << "Printing integer: " << n << std::endl;
6       std::cout << "Printing string: " << str << std::endl;
7   }
8
9   int main() {
10      std::thread t1(print, 10, "T.I.P.");
11      t1.join();
12
13      return 0;
14  }
```

```
PS C:\msys64\ucrt64\bin> & C:\
Printing integer: 10
Printing string: T.I.P.
```

Table 13-2. Multithreaded Example

Code Screenshot	
Output Screenshot	
Analysis	The code implements multithreading by creating different thread objects. These threads are executed simultaneously. The result of running these threads at the same time makes the output in the terminal inconsistent. This can be chalked up to the compiler and the computer managing the prioritization of each thread.

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
0 : T.I.P.3 : Engineers
1 : Competent

2 : Computer
```

```
3 : Engineers
0 : T.I.P.
2 : Computer
1 : Competent
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Table 13-3: Part A Answers

CONDINO

1. **Write a definition of multithreading and its advantages/disadvantages.**
   - Multithreading is the process of dividing tasks into multiple threads that can run concurrently and share the same heap and data segments. This can improve system performance and responsiveness by making threads share CPU, memory, and other resources in a single process. By splitting each task into a separate thread, the task can be done much faster than single-thread processing. However this can also be buggy as they share the same memory address like race conditions; where 2 threads are trying to change the same data at the same time. It is also much harder to code, as one must manage how each thread will handle each resource.
2. **Rationalize the use of multithreading by providing at least 3 use-cases.**
   - It is best to use multithreading when there is a “waiting” period between each task so that other tasks can be accomplished without yielding for these tasks. These tasks I can think of are: web client request management, so that it can handle thousands or sometimes millions of requests concurrently. It is also used when handling user interfaces; you can play a video on a different thread while keeping the main thread of the interface running. Lastly, it can be used for I/O bound operations such as reading and writing files. While one thread yields a response, another can keep working on its task.
3. **Differentiate between parallelism and concurrency.**
   - Concurrency simply means the core manages multiple threads at the same time. Parallelism means splitting the threads across different cores.

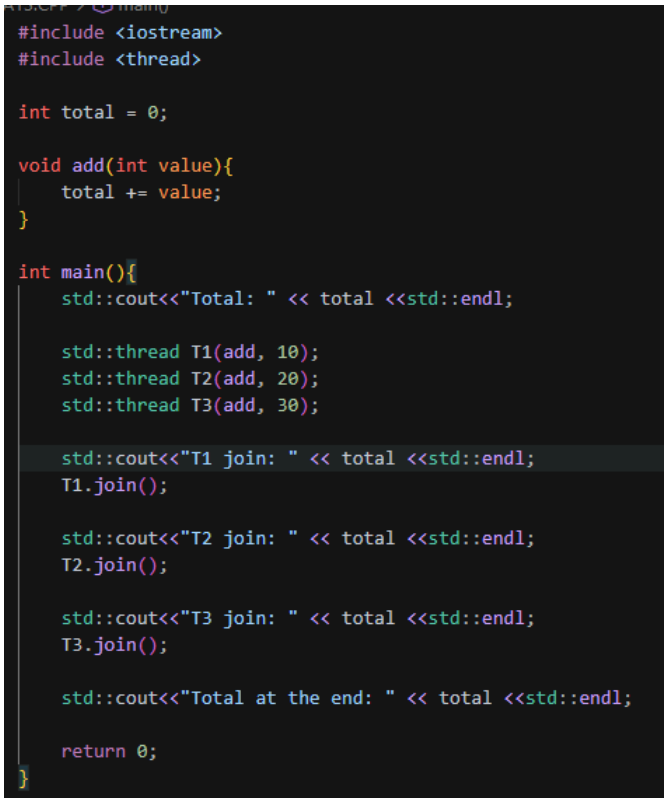
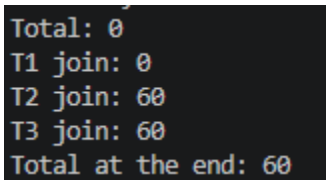
GUARIN

1. **Write a definition of multithreading and its advantages/disadvantages.**
   - Multithreading allows a program to perform multiple tasks using different threads. It can make a program faster and help the computer use its CPU resources better. One disadvantage is that it can also be difficult to manage because threads may access the same data and cause errors if not handled properly.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**
   - Multithreading is useful in different situations, such as file processing, because it allows different files to finish the work faster. In game development, it helps handle tasks like the game logic and audio separately. Lastly, it is also used in web servers to handle requests from different users at the same time, making the server more efficient to use.

3. **Differentiate between parallelism and concurrency.**
   - Concurrency allows multiple tasks to make progress during overlapping periods, even when they are not executing at the same time. Parallelism happens when multiple tasks execute simultaneously, usually on different CPU cores.

Table 13-4: Part B Answers

CODE	
OUTPUT	
ANALYSIS	**CONDINO:** The program creates a new thread but modifies the same variable, so when displaying the value of total when joining each thread, the total jumps from 0 to 60. It did not do it sequentially but added them at the same time, which is why it appears that the summation happened in just "one operation" **GUARIN:** The program creates three threads that add different values to the global variable "total". The join() function makes the main thread wait for a specific thread to finish, but it does not control the order in which the threads execute. The add(0 function updates the total whenever a thread runs. After all the threads have been executed, it resulted in a total of 60.

```cpp
#include <iostream>
#include <thread>

int total = 0;

void add(int value){
    total += value;
}

int main(){
    std::cout<<"Total: " << total <<std::endl;

    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout<<"T1 join: " << total <<std::endl;
    T1.join();

    std::cout<<"T2 join: " << total <<std::endl;
    T2.join();

    std::cout<<"T3 join: " << total <<std::endl;
    T3.join();

    std::cout<<"Total at the end: " << total <<std::endl;

    return 0;
}
```

```
Total: 0
T1 join: 0
T2 join: 60
T3 join: 60
Total at the end: 60
```

PART C.

Code

```cpp
template<typename T>
// the QuickSort function implementation
void quickSortMT(T arr[], int low, int high) {
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        //creating a thread has big runtime cost so we fallback to normal if the size is not too big
        if ((high - low) > 10000){// different threads for each recursive call
            std::thread t1(quickSortMT<T>,arr, low, pi - 1);
            quickSortMT(arr, low: pi + 1, high);//For the main thread
            t1.join();
        }
        else{
            quickSort(arr, low, high: pi - 1);
            quickSort(arr, low: pi + 1, high);
        }
    }
}
```

```cpp
void testTime(int origArr[],void (*sorting)(int*,int,int),std::string name){
    //create deepcopy of orig array
    //prevent stack overflow
    int* copyArr = new int[maxSize];
    for (int i =0;i < maxSize; i++){
        copyArr[i] = origArr[i];
    }

    auto begin: time_point = std::chrono::steady_clock::now();
    sorting(copyArr,0,maxSize-1);
    auto end: time_point = std::chrono::steady_clock::now();

    std::cout << name << " Algorithm's Time: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(d: end-begin).count()
    << " milliseconds\n";

    delete [] copyArr;
}
```

Output

```
~/Downloads/HOA/HOA 13
❯ cd "/home/nielc/Downloads/HOA/HOA 13/" && g++ -std=c++20 Supplementary
Single Threaded Quick Sort Algorithm's Time: 452 milliseconds
Multi Threaded Quick Sort Algorithm's Time: 82 milliseconds
```

Analysis

Condino:

The chosen algorithm in which the multithreading approach is implemented is the quicksort algorithm. This works well with this algorithm because of its divide-and-conquer strategy. Each recursive call has its own thread that is summoned with the aim of making the runtime much better. At first, I naively spawned threads for each call; this hurts the runtime because of the runtime cost of spawning a thread. After some research, I settled on making it fall back to the non-multithreaded version in case the array size is not big enough. I tested this with an array with 1 million elements. Using this testing methodology, the multithreaded version results in a much quicker execution time than the single-threaded version.

Guarin:

The program works by using the quicksort algorithm in both single- and multi - threaded versions; it splits the sorting job into smaller parts so different threads can handle the big ones at the same time. In the multi - threaded version, a new thread sorts the left part while the main thread sorts the right part, and then join() makes the program wait for the thread to finish. We only make a new thread when the part is bigger than 10,000 elements, because making a thread takes time and it's not worth it for small parts. For the small parts, we just use the normal quick sort. To test the speed, we made a testTime function that copies the same unsorted array each time, so both versions sort the same data. In our output, the single-threaded quick sort took 452 milliseconds and the multi-threaded quick sort took 82 milliseconds, which means that the multi-threaded version was about 5.5 times faster. I learned that using threads on the big parts of quick sort can make the program a lot faster.

Conclusion

(Condino)

With this activity, I learned more in-depth concepts about multithreading. Prior to this lab, I already knew the concept of executing different tasks on different cores; however, I was unsure how to implement it. In this activity, I learned about the thread STL of C++ and how it makes the execution of multithreading much easier. I learned that this library by default provides concurrency, but depending on the hardware, it can execute different threads using parallelism. To implement multithreading using this library, it is as simple as creating a thread object and then connecting functions to it. Multithreading has its own trade-offs, so choosing the right time to implement it is crucial. With this knowledge, I plan to use multithreading where it can provide me with the most benefit and where it can lessen my algorithm's runtime.

(Guarin)

In this activity, I gained more knowledge on how to use multithreading in C++. Using the thread library, we were able to create a thread, tell it which function to run, and call join() so the program waits for it to finish. I noticed that the output came out in a different order each time I ran the multithreaded program, since all the threads run together. I also learned that threads can share one variable, but it is prone to errors if they change it at the same time. We used multithreading on QuickSort and observed that it can sort both halves of the array at once. Overall, I think I did well because I understood what each program does, but I'm still not good at writing multithreaded code on my own. I need to practice more with using threads and with checking if multithreading makes a program faster.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.